Architectural Style Justification

Feature Flag & Dynamic Config Manager — Problem Statement #43

1. Architectural Styles Evaluated

Three candidate styles were evaluated against the system's core requirement: pushing flag/config changes to client SDKs in real time, while supporting percentage-based cohort rollouts and environment overrides.

Style	Fit for Real-Time Flag Sync	Strength	Weakness
Layered	Poor — request/response only, no native push mechanism to SDKs	Simple, well understood	Cannot support real-time client sync
Client-Server	Weak — clients must poll for updates, adding latency	Easy to implement	Polling wastes bandwidth, delays rollout
Event-Driven Microservices	Strong — pub/sub pushes flag changes instantly	Real-time sync, independent scaling	Higher operational complexity

2. Selected Style: Event-Driven Microservices

The Event-Driven Microservices style was selected because the platform's defining constraint — real-time client SDK flag synchronization (per Problem Statement #43) — cannot be satisfied by a purely request/response (layered or client-server) style without resorting to inefficient polling. Decomposing the system into independently deployable services (Config Management API, Rule Evaluation Engine, Real-Time Sync Service, Audit & Analytics Logger) also allows the Rule Evaluation Engine — the highest-traffic, latency-sensitive component (NFR-001: <15 ms evaluation) — to scale independently of the lower-traffic Admin Console and Audit Logger, without over-provisioning the whole system.

3. Mapping to the Component Diagram

Each service in the diagram corresponds to a bounded responsibility: the Config Management API exposes IConfigManagement to the Admin Console and depends on IConfigPersistence from the Config Store; the Rule Evaluation Engine exposes IRuleEvaluation, consumed synchronously by the Config Management API and asynchronously (streamed) by the Real-Time Sync Service, which exposes IFlagSync to Client SDKs over a publish/subscribe channel. The Audit & Analytics Logger subscribes to configuration-change events via IAuditLog, keeping audit logging decoupled from the request path so it never adds latency to flag evaluation.

4. Trade-off Accepted

The main cost of this style is increased operational complexity — running and monitoring multiple services and a message broker instead of one monolith. This was accepted because the NFR-001 latency requirement and the real-time sync requirement are non-negotiable for the platform's core use case (safe, fast, targeted feature rollouts), and independent scaling directly supports NFR-002 (security and stability under peak load) by isolating failures to individual services rather than the whole platform.